॥ वसुधैव कुटुम्बकम् ॥

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re-accredited by NAAC with "A" Grade

Name of the Student: Vatsal Postiwala **PRN: 26070122509**

Semester: 3rd **Year: 2025-2029**

Subject Title: Operating Systems Lab	
EXPERIMENT No: 13	
TITLE: Process Synchronization	**DoP**: 19-10-2026

Aim: To Understand Various Process Synchronization techniques.

Learning Outcome:

1. To understand techniques for achieving synchronization between processes

Problem Statements:

Instructions: For each problem, write pseudocode and include appropriate comments explaining the purpose of the semaphore/lock operations and how synchronization is achieved.

1. Producer–Consumer Problem: Restaurant Order Processing System

Concepts: Counting semaphores, mutual exclusion, bounded buffer

Problem statement: A restaurant uses an order processing system in which multiple waiters place customer orders into a shared buffer with a capacity of 10 orders. Multiple kitchen staff members retrieve orders from this buffer and prepare them.

- Waiters must not add orders when the buffer is full.
- Kitchen staff must not retrieve orders when the buffer is empty.
- Only one process may access or modify the shared buffer at a time.
- Concurrent processing must not result in lost, duplicated, or overwritten orders.

Assumption: Multiple producer and consumer processes execute concurrently.

Required task: Write pseudocode for the scenario and add appropriate comments for each major step, especially semaphore/lock initialization, wait/signal operations, critical sections, and prevention of race conditions.

Name:- Posticcoala Vatsal Dipakbhai

E-13 PRN:- 2607012250 9

PAGE NO.

DATE:

1] Producer - Consumer Problem

- Shared Variables & Initialization

```
#define CAPACITY 10

Order buffer [CAPCITY];
int in = 0;
int out = 0;

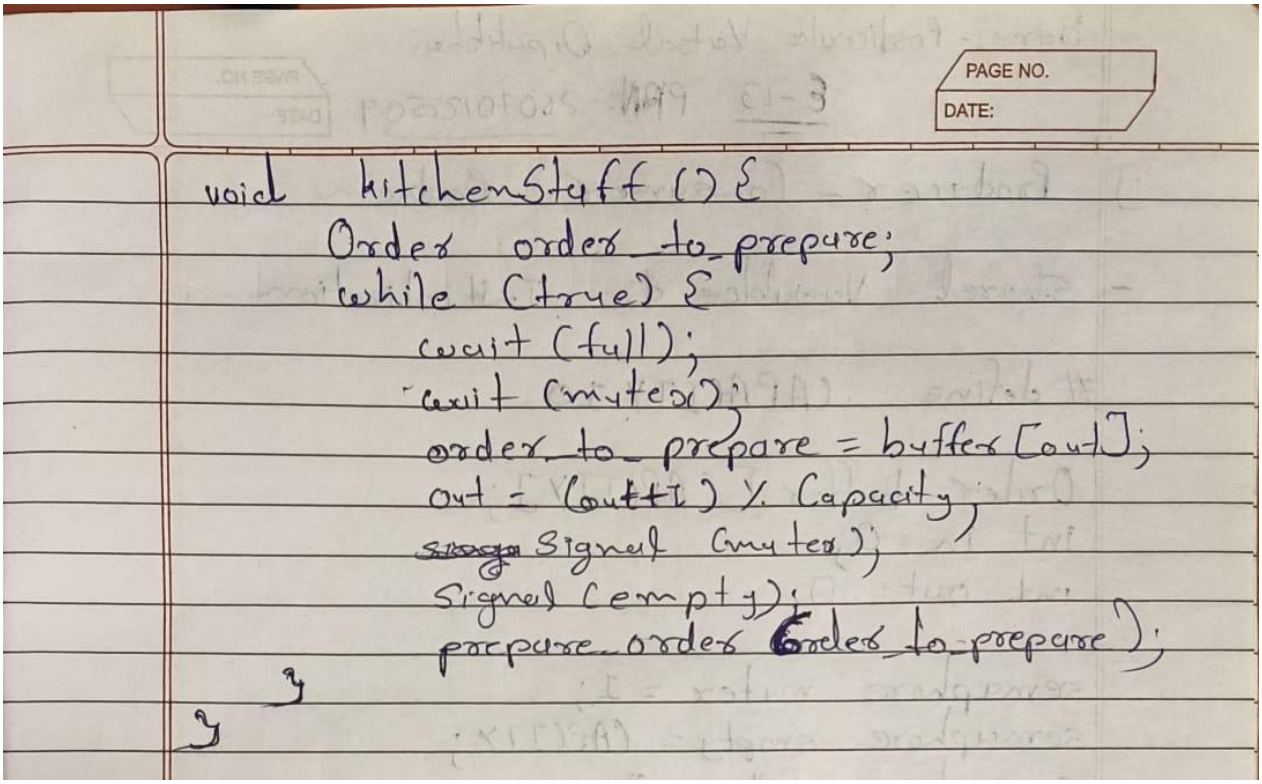
semaphore mutex = 1;
semaphore empty = CAPCITY;
semaphore full = 0;
```

⇒ Pseudocode Implementation

```
void waiter() {
    Order new_order;
    while (true) {
        new_order = take_customer_order();

        wait (empty);

        buffer [in] = new_order;
        in = (in+1) % Capacity;
        Signle (mutex);
        Siagnt
        Signal (mutex);
        Signal (full);
    }
}
```



2. Readers–Writers Problem: Shared Student Database

Concepts: Binary semaphores, reader-count synchronization, writer exclusivity

Problem statement: A university maintains a shared student database accessed concurrently by multiple faculty members. Faculty members may read student records simultaneously, while an administrator may update marks, attendance, or other student details.

- Multiple readers may access the database simultaneously.
- A writer must obtain exclusive access to update records.
- Readers must not access the database while a writer is updating it.
- Database consistency must be maintained during concurrent operations.
- The solution must address the possibility of writer starvation.

Assumption: Use a reader-count variable and appropriate semaphores. State whether the solution prioritizes readers, writers, or fairness.

Required task: Write pseudocode for the scenario and add appropriate comments for each major step, especially semaphore/lock initialization, wait/signal operations, critical sections, and prevention of race conditions.

2) Readers-Writer Problem:-

Database student_db;

int reader_count = 0;

semaphore db_mutex = 1;
semaphore count_mutex = 1;
semaphore service_queue = 1;

=) Pseudocode Implementation:-

```
void Faculty_Reader() {
    while (true) {
        wait(service_queue);
        wait( count_mutex);
        reader_count ++;
        if (reader_count) == 1) {
            wait (db_mutex);
        }
```

```
        signal (count_mutex);
        signal (service_queue);
        read_student_records (student_db);
        wait (count_mutex);
        reader_count--;
        if (reader_count == 0){
            signal (db_mutex);
        }
        signal (count_mutex);
        evaluate_gardes();
    }
}

void Administrator_Writer(){
    Record update_data;
    while (true) {
        update_data = prepare_record_update
                                    ();
        wait (service_queue);
        wait (db_mutex);
        signal (service_queue);
        write_student_records(Student_db,
                              update_data);
        signal (db_mutex);
        log_audit_trail();
    }
}
```

3. Test-and-Set Lock: Concurrent Bank Account Transactions

Concepts: Atomic test-and-set, spinlock, critical section, mutual exclusion

Problem statement: A banking application processes transactions from multiple concurrent threads. Several threads may attempt to update the same bank account balance simultaneously. Without synchronization, concurrent deposits and withdrawals may cause race conditions and incorrect balances.

- Only one thread may enter the critical section at a time.
- The account balance must be updated safely.
- Threads attempting to enter a locked critical section must wait until the lock becomes available.
- The lock must be released after the transaction is completed.
- Identify busy-waiting behavior and the possibility of starvation.

Assumption: TestAndSet() is an atomic hardware-supported operation that returns the previous lock value and sets the lock to true.

Required task: Write pseudocode for the scenario and add appropriate comments for each major step, especially semaphore/lock initialization, wait/signal operations, critical sections, and prevention of race conditions.

PAGE NO.
DATE:

3) Test & Set Lock

```
bool TestAndSet(bool *target) {
    bool initial_value = *target;
    *target = true;
    return initial_value;
}

double account_balance = 50000.00;
bool lock = false;
```

⇒ Pseudocode Implementation:-

```
void ProcessTransaction(Transaction tx) {
    while (true) {
        while (TestAndSet(&lock)) {
        }
        if (tx.type == Deposit) {
            account.balance += tx.amount;
        } else if (tx.type == Withdrawal)
        {
            if (account_balance >= tx.amount)
            {
                account_balance -= tx.amount;
            }
        }
        lock = false;
        generate_receipt(tx);
        break;
    }
}
```

General Submission Guidelines

- Identify the shared resources and concurrent processes/threads.
- Initialize all required semaphores or lock variables correctly.
- Write pseudocode for each process or thread involved.
- Add appropriate comments explaining each major step and synchronization operation.
- Explain how mutual exclusion and synchronization are achieved.
- Identify possible race conditions, deadlocks, or starvation where applicable.
- Clearly state the assumptions and limitations of the proposed solution.

Conclusion:

Through this experiment, I learned how classical synchronization problems arise when multiple processes access shared resources concurrently without coordination. I understood the practical application of binary and counting semaphores to prevent race conditions while managing bounded buffers and starvation-free resource sharing. Finally, implementing hardware-level atomic operations like Test-and-Set demonstrated how mutual exclusion is enforced at the architecture level despite the CPU overhead of busy-waiting.